

Midterm 1 Take-Home

Consider a car of mass $m = 1000\text{kg}$, suspension damping coefficient $b = 4000\text{kg/s}$, and stiffness $k = 20000\text{N/m}$ as shown in figure 1. Consider the road surface shown as a displacement input of the base $u(t)$. Neglect the mass and stiffness of the tire (unsprung mass and stiffness is zero). Assume $x(t)$ is measured from static equilibrium.

Note that, by my sign convention, the positive $\hat{y}$ points directly upward, and the positive $\hat{x}$ direction lies directly to the right.

In part (a), we want to determine the transfer function $(X(s)/U(s))$. First, after applying Newton's second law, we get:

$$\sum F_y = m\ddot{x} = -k(x - u) - b(\dot{x} - \dot{u}) \quad (1)$$

Rearranging Equation 1 yields:

$$m\ddot{x} + b\dot{x} + kx = b\dot{u} + ku \quad (2)$$

If you were to take the Laplace Transform of both sides of Equation 2, we get Equation 3 (assuming zero initial conditions):

$$ms^2X(s) + bsX(s) + kX(s) = X(s)(ms^2 + bs + k) = U(s)(bs + k) = bsU(s) + kU(s) \quad (3)$$

Considering the form of Equation 3 where $X(s)$ and $U(s)$ are factored out of the equation, we can find the transfer equation by divide both sides of the equation by $U(s)$ and $ms^2 + bs + k$, hence:

$$\frac{X(s)}{U(s)} = \frac{bs + k}{ms^2 + bs + k} \quad (4)$$

Now consider $(u(t) = (0.01m) \sin(\omega_b t))$. If we were to take the Laplace Transform of u , we get:

$$U(s) = (0.01m) \left(\frac{\omega_b}{s^2 + \omega_b^2} \right) \quad (5)$$

Plugging Equation 5 into Equation 4 and multiplying both sides by $U(s)$ yields the similar:

$$X(s) = \frac{100(s^2 + \omega_b^2)(bs + k)}{\omega_b(ms^2 + bs + k)} \quad (6)$$

This looks very ugly! Let's use sympy (see reference code in Appendix Code) to solve for the closed form of $x(t)$:

$$x(t) = \frac{\omega(4\omega^2 \cos(4t) + (3\omega^2 - 100) \sin(4t)) - 4(\omega^3 \cos(\omega t) + (\omega^2 - 100) \sin(\omega t))}{100(\omega^4 - 24\omega^2 + 400)} \quad (7)$$

On the following page, we provide the plots/tables for parts (A)-(D). Note that all plots were generated using matplotlib.

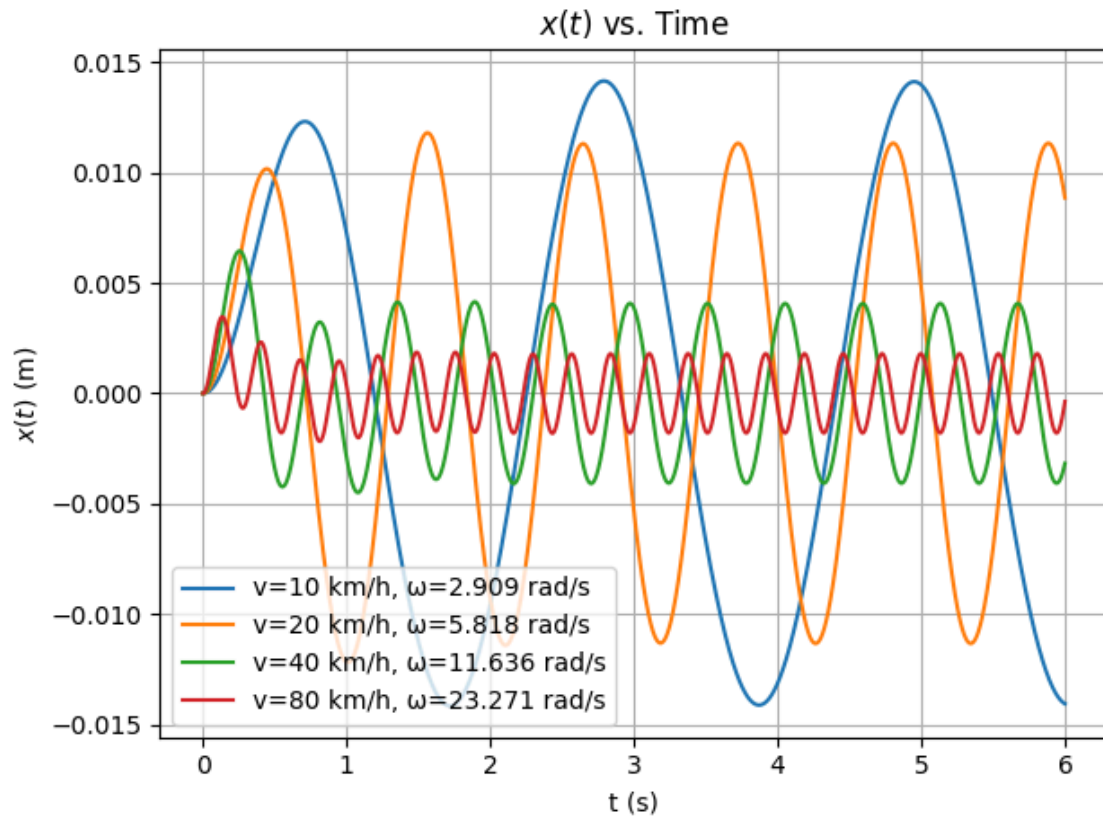


Figure 1: $x(t)$ vs. Time t (Part A)

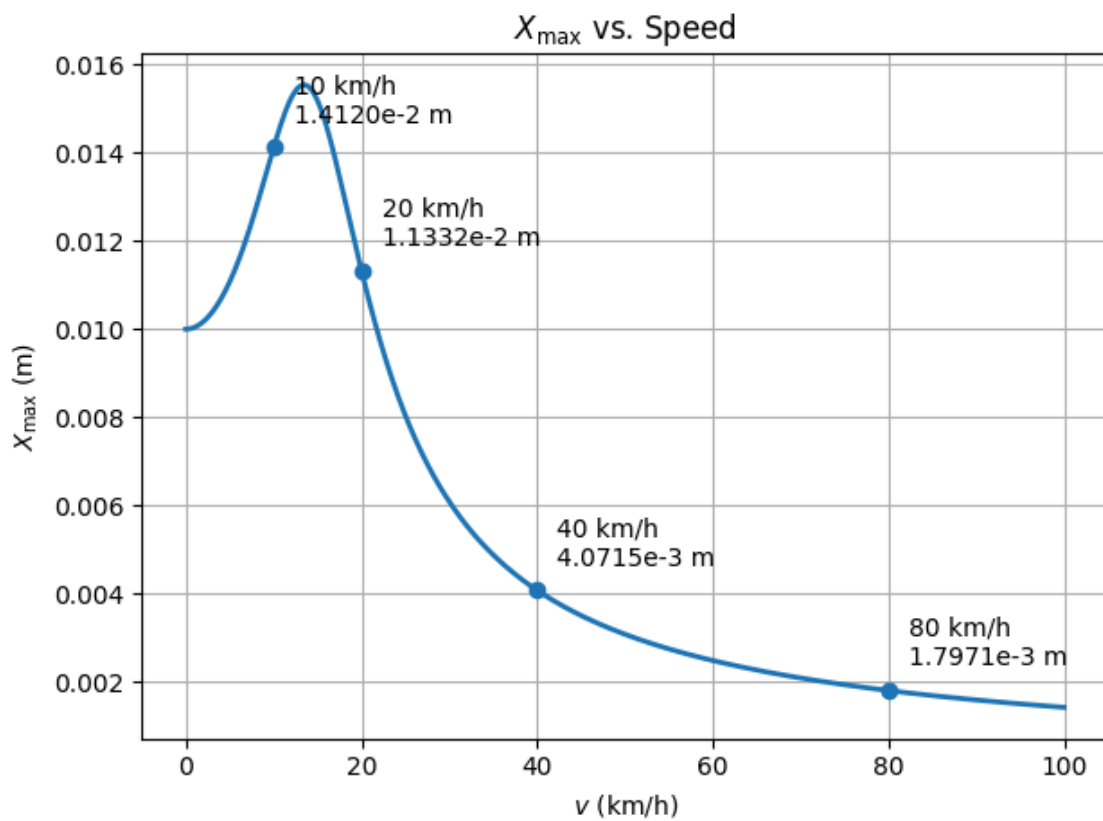


Figure 2: $X_{\max}$ vs. Speed v (Part B)

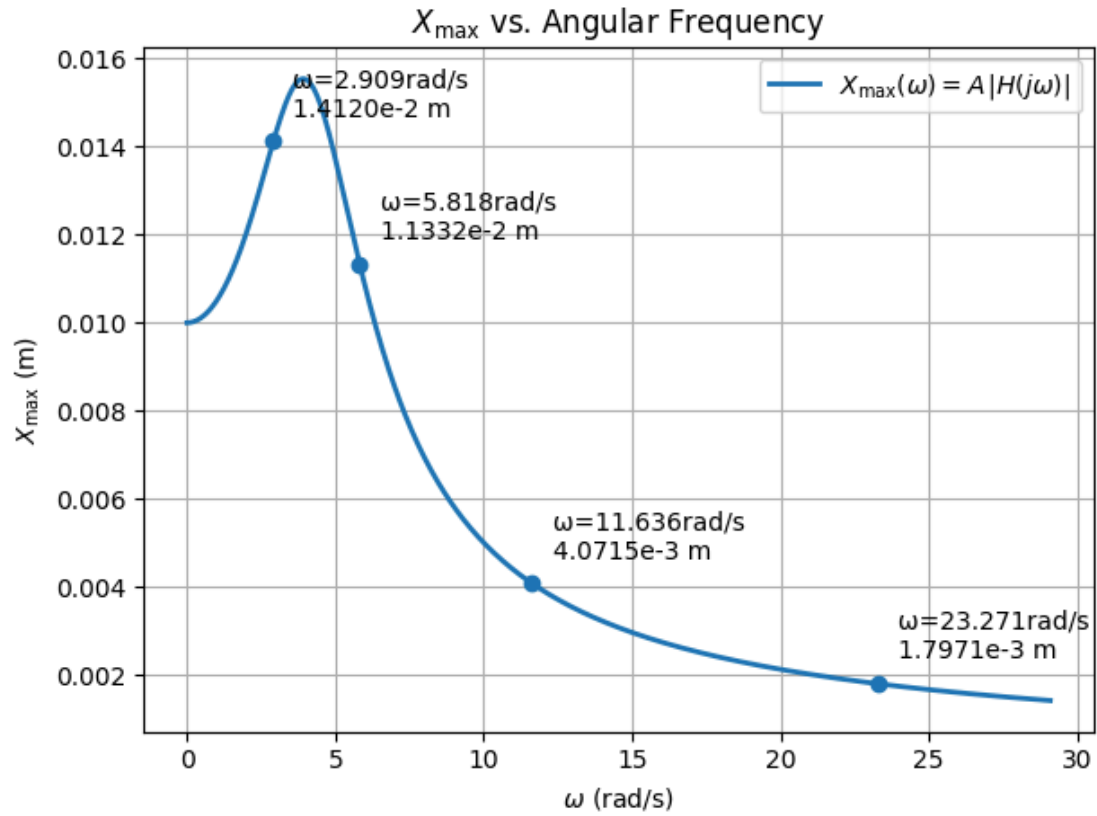


Figure 3: $X_{\max}$ vs. Angular Frequency ω (Part C)

Speed (km/h)	Ang. Freq. (rad/s)	$X_{\max}$ (m)
10	2.909	$1.412 * 10^{-2}$
20	5.818	$1.133 * 10^{-2}$
40	11.636	$4.072 * 10^{-3}$
80	23.271	$1.797 * 10^{-3}$

Table 1: Maximum Displacements for 4 Vehicle Speeds and Corresponding Ang. Frequencies (Part D)

Out of all possible frequencies, $\max(x(t)) = 1.554 * 10^{-2}$ m with corresponding $v = 13.52$ km/h and $\omega = 3.936$ rad/s.

Appendix Code

Here is the following python code used to solve for a closed form of $x(t)$ and plot all of the graphs:

```
import sympy as sp
import numpy as np
import matplotlib.pyplot as plt

# parameters
m_val = 1000 # kg
b_val = 4000 # N*s/m (kg/s)
k_val = 20000 # N/m
A_val = sp.Rational(1, 100) # m

def x_t_expression():
    # symbols
    t, s = sp.symbols("t s", real=True)
    omega = sp.symbols("omega", positive=True, real=True) # forcing freq (rad/s)
    v_sym = sp.symbols("v", positive=True, real=True) # speed parameter (for
    omega_b)

    # transfer function: X/U = (b s + k)/(m s^2 + b s + k)
    H = (b_val * s + k_val) / (m_val * s**2 + b_val * s + k_val)

    # input: u(t) = A sin(omega t) -> U(s) = A*omega/(s^2 + omega^2)
    U = A_val * omega / (s**2 + omega**2)

    # output in Laplace domain and inverse Laplace
    X = sp.simplify(H * U)
    x_t_omega = sp.simplify(sp.inverse_laplace_transform(X, s, t))

    # remove implicit Heaviside(t) for t > 0
    x_t_omega = x_t_omega.subs(sp.Heaviside(t), 1)

    # omega_b(v) symbolically: omega_b = (5*pi/54)*v
    omega_b_sym = sp.Rational(5, 54) * sp.pi * v_sym
    x_t_v = sp.simplify(x_t_omega.subs(omega, omega_b_sym))

    return t, omega, v_sym, x_t_omega, x_t_v

if __name__ == "__main__":
    t_sym, omega_sym, v_sym, x_t_omega, x_t_v = x_t_expression()

    print("\nClosed-form x(t) in terms of omega (rad/s):")
    print(x_t_omega)

    # compute omega_b_values
    v_values = np.array([10.0, 20.0, 40.0, 80.0]) # (use whatever units your formula
    assumes)
    omega_b_values = (5.0 * np.pi / 54.0) * v_values # rad/s

    # time
    t_vals = np.linspace(0.0, 6.0, 10000)

    # change equation syntax to numpy syntax
    x_num = sp.lambdify((t_sym, omega_sym), x_t_omega, modules="numpy")
```

```

v_scan = np.linspace(0.0, 100.0, 600) # km/h (choose range/
resolution you want)
omega_scan = (5.0 * np.pi / 54.0) * v_scan # rad/s

# Evaluate on a (Nt x Nv) grid via broadcasting
# t_vals[:, None] is (Nt,1), omega_scan[None,:] is (1,Nv) -> broadcast to (Nt,Nv)
Y = x_num(t_vals[:, None], omega_scan[None, :])
Y = np.asarray(Y, dtype = float)

# choose X_max
absY = np.abs(Y)
it, iv = np.unravel_index(np.argmax(absY), absY.shape)

t_star = t_vals[it]
v_star = v_scan[iv]
omega_star = omega_scan[iv]
x_star = Y[it, iv]

print("\nGlobal peak over t in [0, 6] s and v in linspace:")
print(f"  max |x(t)| = {abs(x_star):.6e} m  (x = {x_star:+.6e} m)")
print(f"  at t = {t_star:.4f} s,  v = {v_star:.2f} km/h,  ω = {omega_star:.4f}
rad/s")

# plot for part (a)
plt.figure()
for v, w in zip(v_values, omega_b_values):
    y = x_num(t_vals, float(w))
    y = np.asarray(y, dtype=float)
    plt.plot(t_vals, y, label=f"v={v:g} km/h, ω={w:.3f} rad/s")

plt.title("$x(t)$ vs. Time")
plt.xlabel("t (s)")
plt.ylabel("$x(t)$ (m)")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

def H_mag(omega_arr: np.ndarray) -> np.ndarray:
    omega_arr = np.asarray(omega_arr, dtype=float)
    num = np.sqrt(k_val**2 + (b_val * omega_arr)**2)
    den = np.sqrt((k_val - m_val * omega_arr**2)**2 + (b_val * omega_arr)**2)
    return num / den

# Smooth curve over speed
v_plot = np.linspace(0.0, 100.0, 600) # km/h
omega_plot_from_v = (5.0 * np.pi / 54.0) * v_plot
Xmax_vs_v = A_val * H_mag(omega_plot_from_v)

# highlight requested speeds
v_hi = np.array([10.0, 20.0, 40.0, 80.0])
omega_hi = (5.0 * np.pi / 54.0) * v_hi
Xmax_hi = A_val * H_mag(omega_hi)

# plot for part (b)
plt.figure()

```

```

plt.plot(v_plot, Xmax_vs_v, linewidth=2, label=r"$X_{\max}(v)=A\,|H(j\omega_b(v))|$")
plt.scatter(v_hi, Xmax_hi, zorder=5)

for vv, xx in zip(v_hi, Xmax_hi):
    plt.annotate(f"{vv:g} km/h\n{xx:.4e} m",
                 xy=(vv, xx),
                 xytext=(8, 10),
                 textcoords="offset points")

plt.title(r"$X_{\max}$ vs. Speed")
plt.xlabel("$v$ (km/h)")
plt.ylabel(r"$X_{\max}$ (m)")
plt.grid(True)
plt.tight_layout()
plt.show()

# Smooth curve over frequency
omega_plot = np.linspace(0.0, 1.25 * omega_hi.max(), 800) # rad/s
Xmax_vs_omega = A_val * H_mag(omega_plot)

# plot for part (c)
plt.figure()
plt.plot(omega_plot, Xmax_vs_omega, linewidth=2, label=r"$X_{\max}(\omega)=A\,|H(j\omega)|$")

# highlight the base frequencies for the requested speeds
plt.scatter(omega_hi, Xmax_hi, zorder=5)

for vv, ww, xx in zip(v_hi, omega_hi, Xmax_hi):
    plt.annotate(f"$\omega={ww:.3f}$ rad/s \n{xx:.4e} m",
                 xy=(ww, xx),
                 xytext=(8, 10),
                 textcoords="offset points")

plt.title(r"$X_{\max}$ vs. Angular Frequency")
plt.xlabel(r"$\omega$ (rad/s)")
plt.ylabel(r"$X_{\max}$ (m)")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

```

the script returns the following:

```

(omega*(4*omega**2*cos(4*t) + (3*omega**2 - 100)*sin(4*t)) - 4*(omega**3*cos(omega*t)
+ (omega**2 - 100)*sin(omega*t))*exp(2*t))*exp(-2*t)/(100*(omega**4 - 24*omega**2 +
400))

```

Global peak over t in $[0, 6]$ s and v in linspace:

```

max |x(t)| = 1.554239e-02 m (x = +1.554239e-02 m)
at t = 5.3501 s, v = 13.52 km/h,  $\omega$  = 3.9335 rad/s

```